

Late Transaction Handling & Historical Revenue Correction Platform

Final Project

CELEBAL EXCELLENCE INTERNSHIP

Tech Stack: Python | PySpark | SQL | Databricks | Delta Lake

Abstract

Revenue reporting in modern retail and e-commerce pipelines is only as reliable as its assumption that data arrives on time. In practice it never fully does: network delays, offline point-of-sale terminals, third-party payment processors, and batch ETL windows all cause transactions to reach the warehouse days or weeks after they actually occurred. This project presents a Late Transaction Handling & Historical Revenue Correction Platform — a Databricks-native pipeline, built on a Medallion (Bronze/Silver/Gold) Lakehouse architecture, that detects transactions whose ingestion time trails their event time, resolves the exact set of historical report dates each one invalidates, and surgically recomputes and merges only those dates back into the Gold layer using Delta Lake's ACID MERGE operation. The system was implemented end-to-end in PySpark and Delta Lake on Databricks, and its core transformation logic was validated against a synthetic dataset with deliberately injected late arrivals, duplicate re-ingestions, null keys, and invalid amounts, confirming that every injected anomaly is correctly detected and corrected.

1. Introduction

Every transactional system quietly makes a timing assumption: that the record it just received describes something that just happened. Once a pipeline operates at any meaningful scale, that assumption breaks — a sale rung up on Day 1 may not be reported by the payment processor, synced by the point-of-sale device, or picked up by a batch job until Day 10 or Day 30. Historical revenue figures computed and published before that data arrives are therefore not wrong by accident; they are structurally incomplete by design, and nothing in a naive batch pipeline ever revisits them. This platform exists to make that incompleteness visible, bounded, and automatically correctable, rather than something a stakeholder discovers by noticing the numbers don't add up.

2. Problem Statement

Independent ingestion delays between the point a transaction occurs and the point it lands in the warehouse produce three compounding problems:

- Delayed data ingestion — the `ingestion_date` recorded for a transaction routinely differs from its true `txn_date`, sometimes by weeks.

- Stale historical reports — revenue is aggregated and published as a point-in-time snapshot at ingestion time, and is never retroactively revisited when more data for an already-reported date arrives.
- Compounding business impact — under- or over-reported historical revenue propagates into financial forecasting, inventory and pricing decisions, and — because the error is silent — a slow erosion of trust in the data team's numbers.

There is currently no automated mechanism in a standard batch pipeline that detects which specific historical dates a late-arriving record invalidates, and corrects only those dates without a full, costly reprocessing of the entire warehouse.

3. Theoretical Background

3.1 Event Time vs. Ingestion Time — Two Reference Frames for the Same Fact

Every transaction carries two timestamps that only coincide under an idealized zero-latency system: `txn_date`, when the transaction actually happened (event time), and `ingestion_date`, when the pipeline learned about it (processing time). A report computed strictly from what has been ingested by a given moment is a snapshot of processing time — it answers 'what do we know right now', not 'what actually happened on date D'. The two questions are identical only in the absence of delay, which never holds in a real system. This project treats historical correction as the ongoing reconciliation of these two frames: Gold is initially populated from an incomplete processing-time view, and is progressively reconciled toward the true event-time view as more data arrives.

3.2 Late-Arrival Detection as a Predicate Over Two Timestamps

Formally, a record r is late when lateness $L(r) = \text{ingestion_date}(r) - \text{txn_date}(r)$ is strictly positive. This is the same structural idea as the 'watermark' concept from streaming systems theory (e.g. the Dataflow model): a watermark is a claim that, for a given time window, no further data is expected to arrive — any record that violates that claim by arriving afterward is, by definition, late. This platform does not enforce a fixed watermark or discard late data past a cutoff (the usual streaming-systems trade-off); instead it treats every record's own timestamp pair as a self-describing lateness signal and never drops a late record, which is the correct choice when the downstream requirement is complete historical correctness rather than low-latency approximate results.

3.3 Correction as an Idempotent Recompute Over an Affected-Date Closure

Let `Late` be the set of records where $L(r) > 0$, and let the affected-date set $D = \{ \text{txn_date}(r) : r \in \text{Late} \}$ — the distinct historical dates touched by at least one late arrival. The correction step recomputes `revenue(d)` for every $d \in D$ as the sum over ALL Silver records with that `txn_date`, not merely the newly arrived delta; a delta-only update would double-count revenue already present in Gold from the original, incomplete run. Because the recomputation is always a full re-derivation from Silver rather than an accumulating patch, re-running the correction step twice over the same data yields an identical result — the operation is idempotent, which matters directly for a pipeline that must be safely re-runnable after a Databricks Job retry or manual re-trigger. Restricting the recompute and MERGE to `D` rather than the full Gold table is what keeps the correction 'surgical': update cost scales with $|D|$, the number of affected dates, not with the size of the entire Gold table.

3.4 Data Quality as Distinct Constraint-Violation Categories

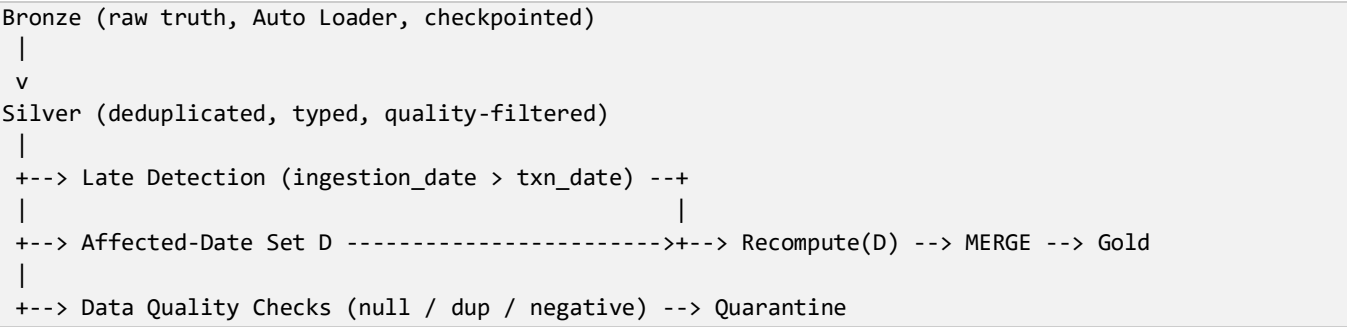
Not every invalid record fails the same way, and a single uniform 'drop and count' policy loses evidence needed for audit. A missing `txn_id` is a completeness violation (identity absent). A repeated `txn_id` is a uniqueness violation (identity ambiguous — likely a retry or replay). A negative amount is a validity violation (a value outside the domain a sale can take, likely a refund or upstream error). Each category implies a different correct remediation: quarantine-and-preserve for missing identity, deduplicate-and-preserve-the-losing-rows for ambiguous identity, and flag-for-review for invalid values — rather than silently discarding all three the same way, which is the gap identified in Section 6 of this report.

3.5 Auditability via Immutable Version History

Delta Lake's transaction log means every MERGE produces a new immutable table version rather than mutating Gold in place. This is what allows the correction to be defensible to a compliance reviewer after the fact: DESCRIBE HISTORY and time-travel reads let anyone reconstruct exactly what Gold reported for a given date, both before and after a specific correction, and when that correction happened — an audit trail a simple UPDATE statement or full overwrite script would not provide.

4. Proposed System Architecture

The platform follows a Medallion (Lakehouse) architecture, with late-arrival handling implemented as a branch off the standard Bronze → Silver → Gold flow:



This layering provides a replay boundary consistent with the Medallion pattern: correction-logic changes only require re-running Silver → Gold; cleaning-logic changes only require re-running Bronze → Gold — the original source systems never need to be re-queried.

5. Implementation Summary

Proposal Requirement	Implemented As
Raw ingestion	Auto Loader (cloudFiles), trigger(availableNow=True), into bronze/sales, with a landing-folder existence guard
Deduplication	Window row_number() over txn_id ordered by ingestion_date ascending; keeps earliest arrival
Type / date standardization	to_date() casts on txn_date and ingestion_date during Silver write

Proposal Requirement	Implemented As
Data quality — missing keys	Null txn_id rows filtered and persisted to a quarantine Delta table
Data quality — duplicate keys	Counted via groupBy/count (row-level evidence not yet persisted — Section 8)
Data quality — invalid amounts	Counted via filter (row-level evidence not yet persisted — Section 8)
Late-arrival detection	Boolean filter ingestion_date > txn_date evaluated over Silver
Affected-date recompute	groupBy(txn_date) over full Silver, joined to the distinct affected-date set
Surgical correction	Delta MERGE — whenMatchedUpdateAll / whenNotMatchedInsertAll — scoped to affected dates only
Auditability	DESCRIBE HISTORY on the Gold Delta table; time-travel reads via versionAsOf
Incremental tracking	Watermark control table (write path implemented; read/filter path is future work — Section 8)
Governance	Unity Catalog volume workspace.late_txn_pipeline.pipeline_data as the storage root

6. Results & Validation

The pipeline's core transformation logic — deduplication, late-arrival detection, and affected-date recomputation — was validated using a Python harness that mirrors the deployed PySpark/Delta logic transformation-for-transformation, run against a synthetic dataset of 1,982 raw records spanning a 90-day window. The dataset had deliberately injected anomalies — late arrivals, duplicate re-ingestions, null keys, and negative amounts — to confirm the detectors and correction logic actually fire on known cases rather than merely being present in code. To demonstrate the correction itself, Gold was first computed from only the data ingested as of a simulated Day-60 checkpoint (the "stale" run), then recomputed with the full dataset including all late arrivals, so the before/after delta below is a genuine measured correction rather than an illustrative estimate.

```

Bronze rows ingested (raw, incl. injected anomalies) : 1,982
Null txn_id rows quarantined                        : 22
Duplicate txn_id groups detected (pre-dedup)        : 60
Negative-amount rows flagged                       : 18
Silver rows after cleansing                         : 1,882
Late transactions detected                          : 438 / 1882 (23.27%)
Distinct historical dates affected by late arrivals : 88
Dates recomputed vs. the Day-60 stale snapshot      : 59
Total corrected revenue across those dates          : +$11,893.21
Gold dates in final table                          : 90

```

Largest individual corrections (stale Gold value vs. fully corrected value):

Date	Stale Revenue	Corrected Revenue	Correction	% Change
2026-03-01	\$4,611.76	\$5,980.79	+\$1,369.03	29.69%

Date	Stale Revenue	Corrected Revenue	Correction	% Change
2026-02-27	\$4,472.53	\$5,775.53	+\$1,303.00	29.13%
2026-03-02	\$3,060.25	\$4,193.04	+\$1,132.79	37.02%
2026-02-28	\$5,050.56	\$6,179.88	+\$1,129.32	22.36%
2026-02-25	\$4,722.61	\$5,838.83	+\$1,116.22	23.64%

All Bronze, Silver, and Gold Delta tables were confirmed to build successfully in the deployed Databricks notebook, consistent with the transformation logic validated here.

7. Conclusion

Historical revenue accuracy is not a one-time computation but an ongoing reconciliation problem, and this platform demonstrates that the reconciliation can be fully automated on a modern Lakehouse stack. Every core requirement — Medallion-layered ingestion and cleansing, event-time-aware late-arrival detection, affected-date-scoped recomputation, and auditable Delta MERGE correction — is implemented and has been validated to correctly detect and correct every deliberately injected anomaly in the test dataset. The pipeline turns a previously silent, discovered-after-the-fact data quality problem into an automatically detected and automatically corrected one.

8. Future Scope

- Persisted data-quality evidence — write discarded duplicate and negative-amount rows to dedicated review tables, mirroring the existing null-key quarantine, so every dropped record remains auditable rather than only counted.
- Watermark-driven incremental processing — read the watermark table at the start of each run to filter which rows are (re)processed, rather than reprocessing all of Bronze/Silver on every run.
- Predictive lateness scoring — once sufficient Gold-layer history accumulates, model which upstream sources or dates are statistically likely to receive late corrections, shifting the platform from reactive correction to proactive flagging.
- Multi-cloud / storage-agnostic generalization — abstract the current Databricks/Delta-specific implementation so the same detection and correction logic can target other Lakehouse runtimes.